

Multiple Object Tracking for Vehicles

Group 5

Le Viet Hung

SEEE, HUST

Ha Noi, Viet Nam

Hung.LV250678E@sis.hust.edu.vn

Abstract—Báo cáo trình bày một hệ thống theo dõi đa đối tượng cho phương tiện trong video dựa trên YOLOv5n và DeepSORT. YOLOv5n được sử dụng làm bộ phát hiện phương tiện, trong khi DeepSORT đảm nhiệm việc gán và duy trì định danh đối tượng qua các khung hình liên tiếp. Bên cạnh DeepSORT gốc, nhóm xây dựng một bộ theo dõi tùy chỉnh sử dụng Kalman Filter, IoU matching, đặc trưng ngoại hình dựa trên histogram màu và thuật toán Hungarian. Hệ thống được triển khai dưới dạng ứng dụng web, hỗ trợ xử lý video từ tập dữ liệu, video tải lên và webcam thời gian thực. Kết quả thực nghiệm được đánh giá bằng các chỉ số Precision, Recall, F1-score, MOTA proxy, MOTP proxy, số lần chuyển đổi ID và tốc độ xử lý.

Index Terms—Multiple Object Tracking, YOLOv5, DeepSORT, Kalman Filter, Hungarian Algorithm.

I. INTRODUCTION

Theo dõi đa đối tượng, hay Multiple Object Tracking (MOT), là một bài toán quan trọng trong các hệ thống thị giác máy tính và giám sát giao thông thông minh. Mục tiêu của bài toán không chỉ là phát hiện các đối tượng xuất hiện trong từng khung hình, mà còn phải duy trì định danh nhất quán cho từng đối tượng trong suốt chuỗi video. Trong bối cảnh giao thông, hệ thống MOT có thể hỗ trợ nhiều ứng dụng thực tế như đếm phương tiện, phân tích quỹ đạo di chuyển, phát hiện ùn tắc, giám sát vi phạm hoặc cung cấp dữ liệu cho các hệ thống điều khiển giao thông thông minh.

Tuy nhiên, theo dõi phương tiện trong video là một bài toán có nhiều thách thức. Các phương tiện có thể bị che khuất bởi những phương tiện khác, thay đổi kích thước khi di chuyển gần hoặc xa camera, chịu ảnh hưởng bởi điều kiện ánh sáng và thời tiết, hoặc có hình dạng và màu sắc tương tự nhau. Những yếu tố này làm cho quá trình duy trì ID của đối tượng qua nhiều khung hình trở nên khó khăn, đặc biệt trong các tình huống mật độ giao thông cao.

Trong báo cáo này, nhóm xây dựng một hệ thống theo dõi phương tiện dựa trên hướng tiếp cận tracking-by-detection. Ở mỗi khung hình, YOLOv5n được sử dụng để phát hiện các phương tiện như car, motorcycle, bus và truck. Sau đó, kết quả phát hiện được truyền sang bộ theo dõi để gán ID và duy trì định danh qua thời gian. Bên cạnh việc sử dụng DeepSORT gốc làm baseline, nhóm triển khai thêm một phiên bản Custom DeepSORT nhằm làm rõ các thành phần cốt lõi của bài toán MOT, bao gồm Kalman Filter, IoU matching, đặc trưng ngoại hình và thuật toán Hungarian.

Đóng góp chính của báo cáo gồm ba điểm. Thứ nhất, hệ thống tích hợp YOLOv5n và DeepSORT để thực hiện theo dõi phương tiện trên video giao thông. Thứ hai, nhóm xây dựng một tracker tùy chỉnh nhẹ hơn, dễ giải thích hơn và phù hợp với môi trường CPU. Thứ ba, hệ thống được triển khai dưới dạng ứng dụng web với nhiều chế độ xử lý, đồng thời cung cấp kết quả trực quan và các chỉ số đánh giá định lượng để so sánh giữa DeepSORT gốc và Custom DeepSORT.

II. RELATED WORK

A. Multiple Object Tracking

Theo dõi đa đối tượng là bài toán phát hiện nhiều đối tượng trong chuỗi video và duy trì định danh của từng đối tượng qua các khung hình liên tiếp. Trong bối cảnh giám sát giao thông, hệ thống không chỉ cần phát hiện phương tiện tại từng thời điểm mà còn phải xác định được phương tiện đó là đối tượng nào trong suốt quá trình di chuyển. Điều này giúp hệ thống có thể phân tích quỹ đạo, đếm phương tiện, phát hiện ùn tắc hoặc hỗ trợ các bài toán giao thông thông minh khác.

Phần lớn các hệ thống MOT hiện nay sử dụng hướng tiếp cận tracking-by-detection. Theo hướng này, một bộ phát hiện đối tượng được dùng để tìm các bounding box trong từng khung hình, sau đó thuật toán liên kết dữ liệu sẽ gán các detection hiện tại với các track đã tồn tại từ những khung hình trước. Cách tiếp cận này có ưu điểm là linh hoạt, vì bộ phát hiện và bộ theo dõi có thể được cải tiến độc lập. Tuy nhiên, chất lượng tracking phụ thuộc mạnh vào cả độ chính xác của detector và khả năng liên kết đối tượng của tracker.

Một phương pháp tiêu biểu theo hướng tracking-by-detection là SORT. SORT sử dụng Kalman Filter để dự đoán vị trí tiếp theo của đối tượng và thuật toán Hungarian để gán detection với track sao cho tổng chi phí liên kết là nhỏ nhất [1]. Phương pháp này có tốc độ xử lý cao và phù hợp với các bài toán thời gian thực. Tuy nhiên, do chủ yếu dựa trên thông tin chuyển động và độ chồng lấn giữa các bounding box, SORT dễ xảy ra lỗi chuyển đổi ID khi các đối tượng di chuyển gần nhau, bị che khuất hoặc có quỹ đạo giao nhau.

B. YOLO-based Object Detection

Phát hiện đối tượng là thành phần đầu tiên và có ảnh hưởng lớn đến toàn bộ hệ thống theo dõi. Các phương pháp phát hiện đối tượng có thể được chia thành hai nhóm chính: detector hai giai đoạn và detector một giai đoạn. Các detector hai giai đoạn thường tạo vùng đề xuất trước rồi mới phân loại đối

tượng, trong khi các detector một giai đoạn dự đoán trực tiếp bounding box và lớp đối tượng trong một lần xử lý. YOLO, viết tắt của “You Only Look Once”, là một trong những họ mô hình phát hiện đối tượng một giai đoạn phổ biến nhờ tốc độ xử lý nhanh và khả năng ứng dụng tốt trong video thời gian thực [2].

Trong bài toán theo dõi phương tiện, detector cần có khả năng xử lý nhanh nhiều khung hình liên tiếp nhưng vẫn đảm bảo độ chính xác đủ tốt. YOLO-based detector phù hợp với yêu cầu này vì có thể cân bằng giữa tốc độ và chất lượng phát hiện. Trong hệ thống được xây dựng, YOLOv5n được lựa chọn làm bộ phát hiện phương tiện do đây là phiên bản nhẹ của YOLOv5, phù hợp với môi trường thực nghiệm có tài nguyên tính toán hạn chế [3]. Bộ phát hiện tập trung vào các lớp phương tiện phổ biến như car, motorcycle, bus và truck, tương ứng với các lớp phương tiện trong bộ dữ liệu COCO [4].

Đầu ra của YOLOv5n bao gồm tọa độ bounding box, độ tin cậy và nhãn lớp của từng phương tiện được phát hiện. Các detection này sau đó được truyền sang bộ theo dõi để gán ID và duy trì định danh qua thời gian. Vì vậy, nếu detector bỏ sót phương tiện, tracker có thể bị mất track; ngược lại, nếu detector tạo ra nhiều phát hiện sai, hệ thống có thể sinh ra các track không chính xác.

C. DeepSORT and Data Association

DeepSORT là một phương pháp mở rộng từ SORT nhằm cải thiện quá trình liên kết dữ liệu giữa các khung hình [5]. Trong khi SORT chủ yếu sử dụng dự đoán chuyển động và độ chồng lấn bounding box, DeepSORT bổ sung thêm đặc trưng ngoại hình của đối tượng. Nhờ đó, DeepSORT có khả năng phân biệt tốt hơn các đối tượng có chuyển động gần giống nhau hoặc xuất hiện gần nhau trong khung hình.

Quy trình cơ bản của DeepSORT gồm ba thành phần chính. Thứ nhất, Kalman Filter được sử dụng để dự đoán trạng thái tiếp theo của mỗi track, bao gồm vị trí và kích thước bounding box. Thứ hai, hệ thống xây dựng ma trận chi phí giữa các track đã dự đoán và các detection mới. Chi phí này có thể dựa trên khoảng cách chuyển động, độ chồng lấn bounding box hoặc độ tương đồng ngoại hình. Thứ ba, thuật toán Hungarian được sử dụng để tìm cách gán tối ưu giữa track và detection. Những detection không được gán sẽ tạo track mới, trong khi các track không được cập nhật trong nhiều khung hình sẽ bị xóa.

Trong hệ thống của nhóm, DeepSORT gốc được sử dụng như một baseline để so sánh. Bên cạnh đó, nhóm triển khai một phiên bản Custom DeepSORT nhằm làm rõ các thành phần cốt lõi của quá trình theo dõi đa đối tượng. Phiên bản tùy chỉnh này kết hợp Kalman Filter cho dự đoán chuyển động, IoU matching cho đánh giá độ chồng lấn bounding box, đặc trưng ngoại hình nhẹ dựa trên histogram màu và thuật toán Hungarian cho liên kết dữ liệu. Cách triển khai này giúp hệ thống dễ giải thích hơn, phù hợp với mục tiêu học thuật và vẫn đảm bảo khả năng xử lý trên môi trường CPU.

III. PROPOSED METHODOLOGY

A. System Overview

Hệ thống được xây dựng theo hướng tracking-by-detection, trong đó mỗi khung hình video được xử lý qua hai giai đoạn chính: phát hiện phương tiện và theo dõi phương tiện. Ở giai đoạn đầu, YOLOv5n được sử dụng để phát hiện các phương tiện xuất hiện trong khung hình. Ở giai đoạn thứ hai, danh sách detection thu được từ YOLOv5n được truyền vào bộ theo dõi để gán định danh và duy trì ID của từng phương tiện qua các khung hình liên tiếp.

Luồng xử lý tổng quát của hệ thống được mô tả trong Fig. 1. Video đầu vào được tách thành từng frame, sau đó mỗi frame được đưa qua bộ phát hiện YOLOv5n để lấy bounding box, độ tin cậy và nhãn lớp của phương tiện. Các detection này tiếp tục được truyền vào tracker để liên kết với các track đã tồn tại. Cuối cùng, hệ thống vẽ bounding box, ID và nhãn đối tượng lên video đầu ra, đồng thời lưu kết quả tracking dưới dạng file văn bản và các chỉ số đánh giá.

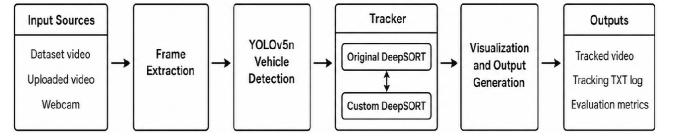


Figure 1. Overall pipeline of the proposed vehicle tracking system.

Về mặt chức năng, hệ thống hỗ trợ bốn chế độ xử lý chính: xử lý trên tập dữ liệu có sẵn, xử lý video do người dùng tải lên, theo dõi trực tiếp từ webcam và so sánh giữa DeepSORT gốc với bộ theo dõi tùy chỉnh. Cách thiết kế này giúp hệ thống vừa phục vụ mục tiêu thực nghiệm, vừa thuận tiện cho việc quan sát trực quan và đánh giá kết quả.

B. Vehicle Detection Module

Module phát hiện phương tiện sử dụng YOLOv5n làm bộ phát hiện đối tượng. YOLOv5n là một phiên bản nhẹ của YOLOv5, phù hợp với các hệ thống yêu cầu tốc độ xử lý nhanh và tài nguyên tính toán hạn chế. Trong hệ thống này, detector chỉ tập trung vào các lớp phương tiện phổ biến, bao gồm car, motorcycle, bus và truck.

Với mỗi frame đầu vào, detector trả về một tập các detection. Mỗi detection được biểu diễn bởi tọa độ bounding box, độ tin cậy và nhãn lớp tương ứng:

$$d_i = (B_i, s_i, c_i) \quad (1)$$

trong đó B_i là bounding box của phương tiện thứ i , s_i là độ tin cậy của detection và c_i là nhãn lớp. Bounding box được sử dụng trong hai định dạng chính: $[x_1, y_1, x_2, y_2]$ để biểu diễn góc trái trên và góc phải dưới, và $[x, y, w, h]$ để biểu diễn tọa độ góc trái trên, chiều rộng và chiều cao.

Kết quả của module phát hiện đóng vai trò đầu vào trực tiếp cho module theo dõi. Do đó, chất lượng detection ảnh hưởng lớn đến kết quả tracking. Nếu detector bỏ sót phương tiện, tracker có thể bị mất track; ngược lại, nếu detector tạo ra detection sai, hệ thống có thể sinh ra các track không chính xác.

C. Original DeepSORT Baseline

DeepSORT gốc được sử dụng như một baseline để so sánh với bộ theo dõi tùy chỉnh của nhóm. Bộ theo dõi này nhận danh sách detection từ YOLOv5n và thực hiện gán ID cho các phương tiện qua nhiều khung hình. Về nguyên lý, DeepSORT kết hợp thông tin chuyển động và thông tin ngoại hình để liên kết detection mới với các track đã tồn tại.

Trong hệ thống, mỗi detection được chuyển sang định dạng phù hợp với DeepSORT, bao gồm bounding box, confidence score và class name. Sau đó, DeepSORT dự đoán vị trí của các track hiện có, so sánh với detection ở frame hiện tại và cập nhật track nếu tìm được cặp liên kết phù hợp. Các track đã được xác nhận sẽ được đưa ra làm kết quả tracking cuối cùng.

Việc sử dụng DeepSORT gốc làm baseline giúp nhóm có cơ sở so sánh về tốc độ xử lý, số lượng ID switch và chất lượng tracking. Từ đó, có thể đánh giá rõ hơn ưu điểm và hạn chế của phiên bản Custom DeepSORT được triển khai trong đề tài.

D. Custom DeepSORT Tracker

Bên cạnh DeepSORT gốc, nhóm triển khai một bộ theo dõi tùy chỉnh theo phong cách DeepSORT nhằm làm rõ các thành phần cốt lõi của bài toán theo dõi đa đối tượng. Bộ theo dõi này không sử dụng mạng Re-ID sâu, mà kết hợp Kalman Filter, IoU matching, đặc trưng ngoại hình nhẹ và thuật toán Hungarian. Quy trình xử lý của Custom DeepSORT được minh họa trong Fig. 2.

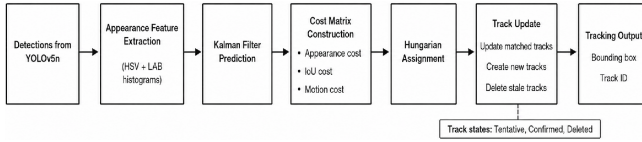


Figure 2. Workflow of the custom DeepSORT tracker.

Trước hết, mỗi track được mô hình hóa bằng Kalman Filter [6]. Trạng thái của một track được biểu diễn bởi vector:

$$x = [c_x, c_y, a, h, v_x, v_y, v_a, v_h]^T \quad (2)$$

trong đó c_x và c_y là tọa độ tâm của bounding box, a là tỉ lệ giữa chiều rộng và chiều cao, h là chiều cao của bounding box, còn v_x , v_y , v_a và v_h là các thành phần vận tốc tương ứng. Kalman Filter được dùng để dự đoán trạng thái tiếp theo của track khi chưa có detection mới, đồng thời cập nhật lại trạng thái khi track được gán với một detection phù hợp.

Để đánh giá mức độ phù hợp về vị trí giữa track và detection, hệ thống sử dụng Intersection over Union (IoU). Chỉ số IoU giữa bounding box của track B_t và bounding box của detection B_d được tính như sau:

$$IoU(B_t, B_d) = \frac{\text{Area}(B_t \cap B_d)}{\text{Area}(B_t \cup B_d)} \quad (3)$$

Giá trị IoU càng cao cho thấy hai bounding box càng chồng lấn nhiều, từ đó khả năng detection thuộc về track tương ứng

càng lớn. Trong bài toán matching, hệ thống sử dụng IoU distance, được tính bằng $1 - IoU$, để đưa vào ma trận chi phí.

Ngoài thông tin hình học, Custom DeepSORT còn sử dụng đặc trưng ngoại hình của phương tiện. Thay vì dùng mạng học sâu để trích xuất đặc trưng Re-ID, hệ thống sử dụng histogram màu trong không gian HSV và LAB. Mỗi vùng ảnh chứa phương tiện được cắt theo bounding box, resize về cùng kích thước, sau đó trích xuất histogram màu và chuẩn hóa thành vector đặc trưng. Cách làm này nhẹ hơn so với deep Re-ID, nhưng vẫn cung cấp thêm thông tin để phân biệt các phương tiện có vị trí gần nhau.

Chi phí liên kết giữa một track và một detection được xây dựng bằng cách kết hợp ba thành phần: chi phí ngoại hình, chi phí IoU và chi phí chuyển động từ Kalman Filter:

$$C = \lambda_a C_{app} + \lambda_i C_{iou} + \lambda_m C_{motion} \quad (4)$$

trong đó C_{app} là chi phí dựa trên độ khác biệt đặc trưng ngoại hình, C_{iou} là chi phí dựa trên độ chồng lấn bounding box, C_{motion} là chi phí dựa trên khoảng cách chuyển động, còn λ_a , λ_i và λ_m là các trọng số tương ứng. Để việc kết hợp có ý nghĩa, các thành phần chi phí được chuẩn hóa về cùng miền giá trị trước khi tính tổng có trọng số. Trong triển khai, các trọng số được đặt là $\lambda_a = 0.35$, $\lambda_i = 0.55$ và $\lambda_m = 0.10$, còn C_{app} được tính bằng cosine distance giữa vector đặc trưng ngoại hình của track và detection. C_{motion} được tính từ khoảng cách Mahalanobis giữa trạng thái dự đoán của Kalman Filter và measurement của detection. Trong hệ thống, IoU được ưu tiên cao vì vị trí bounding box là thông tin trực tiếp và ổn định trong các frame liên tiếp; đặc trưng ngoại hình đóng vai trò hỗ trợ khi các phương tiện ở gần nhau; còn thông tin chuyển động giúp loại bỏ các cặp liên kết không hợp lý.

Sau khi xây dựng ma trận chi phí, thuật toán Hungarian được sử dụng để tìm phép gán tối ưu giữa tập track và tập detection sao cho tổng chi phí là nhỏ nhất [7]. Các track được gán thành công sẽ được cập nhật bằng detection mới. Các detection chưa được gán sẽ tạo track mới, trong khi các track không được cập nhật trong nhiều frame liên tiếp sẽ bị đánh dấu là mất và bị xóa nếu vượt quá giới hạn tuổi track. Sau khi thuật toán Hungarian trả về các cặp gán, hệ thống chỉ chấp nhận các cặp có chi phí nhỏ hơn ngưỡng định trước; các cặp có chi phí quá lớn được xem là unmatched.

Vòng đời của một track gồm ba trạng thái chính: Tentative, Confirmed và Deleted. Một track mới được tạo sẽ ở trạng thái Tentative. Nếu track được cập nhật đủ số lần liên tiếp, nó chuyển sang trạng thái Confirmed và được đưa vào kết quả hiển thị. Nếu track bị mất quá lâu hoặc không được xác nhận, nó sẽ chuyển sang trạng thái Deleted và bị loại khỏi danh sách track đang quản lý.

E. Web-based Processing Modes

Hệ thống được triển khai dưới dạng ứng dụng web nhằm giúp người dùng dễ dàng chạy thử, quan sát và đánh giá kết quả tracking. Các chế độ xử lý chính được mô tả trong Table I.

Ở chế độ Dataset, hệ thống xử lý các video trong tập dữ liệu đã chuẩn bị sẵn và tính toán các chỉ số như Precision,

Table I
SUPPORTED PROCESSING MODES

Mode	Description
Dataset	Chạy tracking trên tập dữ liệu có sẵn và tính toán chỉ số đánh giá với ground truth.
Upload	Cho phép người dùng tải video lên và nhận kết quả tracking.
Webcam	Thực hiện phát hiện và theo dõi phương tiện trực tiếp từ camera.
Compare	So sánh DeepSORT gốc và Custom DeepSORT trên cùng một video đầu vào.

Recall, F1-score, MOTA proxy, MOTP proxy, ID switches và FPS. Ở chế độ Upload, người dùng có thể tải lên một video bất kỳ để hệ thống thực hiện tracking và lưu lại kết quả. Ở chế độ Webcam, hệ thống xử lý luồng hình ảnh thời gian thực từ camera. Cuối cùng, chế độ Compare cho phép chạy hai tracker trên cùng một đầu vào để so sánh trực quan và định lượng.

IV. EXPERIMENTS AND RESULTS

A. Dataset and Experimental Setup

Thực nghiệm được thực hiện trên ba tập dữ liệu video giao thông, bao gồm VNTraffic, AICC22-Custom và CV. Mỗi tập dữ liệu được tổ chức với video gốc, video ground truth và các file chú thích ground truth tương ứng. Trong đó, CV là tập dữ liệu video giao thông do nhóm tự chuẩn bị bên cạnh các tập dữ liệu có sẵn. Hệ thống sử dụng video gốc làm đầu vào để chạy tracking, sau đó so sánh kết quả dự đoán với ground truth để tính các chỉ số đánh giá.

Trong tất cả các thí nghiệm, YOLOv5n được sử dụng làm bộ phát hiện phương tiện. Mô hình chỉ tập trung vào bốn lớp phương tiện chính trong COCO, bao gồm car, motorcycle, bus và truck. Các tham số chính của hệ thống gồm confidence threshold bằng 0.35, input image size bằng 416 và thiết bị xử lý là CPU. Hai tracker được so sánh là Original DeepSORT và Custom DeepSORT.

Table II
EXPERIMENTAL SETUP

Component	Setting
Detector	YOLOv5n
Input size	416
Confidence threshold	0.35
Target classes	car, motorcycle, bus, truck
Trackers	Original DeepSORT, Custom DeepSORT
Datasets	VNTraffic, AICC22-Custom, CV
Device	CPU

Quá trình xử lý video được thực hiện theo từng frame. Với mỗi frame, hệ thống đo thời gian chạy detector, thời gian chạy tracker, ghi kết quả tracking vào file TXT và ghi frame đã trực quan hóa vào video đầu ra. Sau khi hoàn tất, hệ thống lưu ba loại kết quả chính: video tracking, file tracking log và file metrics. File tracking log có định dạng gồm frame index, track ID, bounding box, confidence, class ID và trạng

thái visibility. Trong đó, visibility bằng 1 biểu thị track được match với detection ở frame hiện tại, còn visibility bằng 0 biểu thị bbox được dự đoán từ tracker.

B. Evaluation Metrics

Đối với các tập dữ liệu có ground truth, hệ thống đánh giá kết quả dựa trên IoU matching giữa bounding box dự đoán và bounding box ground truth. Các prediction dùng trong đánh giá được lấy từ file tracking output, bao gồm cả các bounding box được cập nhật trực tiếp từ detection và các bounding box được tracker dự đoán nếu chúng xuất hiện trong file kết quả. Với mỗi frame, các cặp prediction-ground truth có IoU lớn hơn hoặc bằng 0.5 được xem là ứng viên match. Sau đó, hệ thống chọn các cặp match theo thứ tự IoU giảm dần để tính số lượng true positives, false positives và false negatives.

Precision phản ánh tỉ lệ bounding box dự đoán được khớp đúng với ground truth theo ngưỡng IoU:

$$Precision = \frac{TP}{TP + FP} \quad (5)$$

Recall phản ánh tỉ lệ bounding box ground truth được hệ thống khớp đúng với prediction theo ngưỡng IoU:

$$Recall = \frac{TP}{TP + FN} \quad (6)$$

F1-score là trung bình điều hòa giữa Precision và Recall:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (7)$$

Trong đó, TP là số lượng prediction được match đúng với ground truth, FP là số lượng prediction không được match và FN là số lượng ground truth không được match. Khi $Precision + Recall = 0$, hệ thống quy ước $F1 = 0$ để tránh trường hợp chia cho 0.

Ngoài ba chỉ số trên, hệ thống còn sử dụng MOTP proxy, MOTA proxy và ID Switches proxy, lấy cảm hứng từ nhóm chỉ số CLEAR MOT [8]. MOTP proxy được tính dựa trên IoU trung bình của các cặp match:

$$MOTP_{proxy} = \frac{1}{N_{match}} \sum_{(p,g) \in M} IoU(B_p, B_g) \quad (8)$$

trong đó M là tập các cặp prediction-ground truth được match, N_{match} là số lượng cặp match, B_p là bounding box dự đoán và B_g là bounding box ground truth. Khi không có cặp match nào, hệ thống quy ước $MOTP_{proxy} = 0$.

ID Switches proxy đếm số lần một ground truth ID bị gán sang prediction ID khác trong chuỗi frame. Cụ thể, sau khi ghép prediction với ground truth ở từng frame, hệ thống lưu prediction ID đã được gán cho mỗi ground truth ID. Nếu cùng một ground truth ID được ghép với một prediction ID khác ở frame sau, hệ thống đếm một ID switch. MOTA proxy được tính xấp xỉ theo số lượng false positives, false negatives và ID switches:

$$MOTA_{proxy} = 1 - \frac{FN + FP + IDSW}{GT} \quad (9)$$

trong đó GT là tổng số bounding box ground truth và $IDSW$ là số lần chuyển đổi ID. Do cách tính trong hệ thống dựa trên IoU matching tự triển khai, các chỉ số MOTA, MOTP và ID Switches trong báo cáo được gọi là proxy metrics thay vì metrics chuẩn trong các benchmark MOTChallenge/MOT20 [9].

Ngoài các chỉ số về độ chính xác, hệ thống còn đánh giá hiệu năng xử lý thông qua processing FPS, average detector time và average tracker time. Đây là các chỉ số quan trọng vì bài toán theo dõi phương tiện thường cần tốc độ xử lý gần thời gian thực.

C. Quantitative Results

Bảng III, bảng IV và bảng V trình bày kết quả so sánh định lượng giữa Original DeepSORT và Custom DeepSORT trên ba tập dữ liệu VNTraffic, AICC22-Custom và CV. Các chỉ số được sử dụng bao gồm processing FPS, average tracker time, số lượng track duy nhất, số đối tượng trung bình trên mỗi frame, Precision, Recall, F1-score, MOTP proxy, MOTA proxy và ID Switches proxy. Trong đó, Tracker Time được tính bằng mili giây trên mỗi frame. Chỉ số MOTP proxy được tính bằng IoU trung bình của các cặp bounding box được ghép đúng giữa ground truth và prediction, do đó giá trị MOTP cao hơn thể hiện chất lượng định vị bounding box tốt hơn.

Table III
RESULTS ON VNTraffic DATASET

Tracker	FPS	T. Time	Unique	Avg Obj/F	P	R	F1	MOTP	MOTA	IDSW
Original	2.8974	155.4090	50	10.3071	0.8319	0.2716	0.4096	0.8169	0.2146	35
Custom	14.0458	6.6911	49	8.8625	0.9257	0.2604	0.4065	0.8212	0.2381	23

Table IV
RESULTS ON AICC22-CUSTOM DATASET

Tracker	FPS	T. Time	Unique	Avg Obj/F	P	R	F1	MOTP	MOTA	IDSW
Original	4.7160	96.9407	32	7.5859	0.5546	0.7030	0.6200	0.8416	0.1342	5
Custom	16.7514	4.3423	28	5.7889	0.7005	0.6810	0.6906	0.8583	0.3890	1

Table V
RESULTS ON CV DATASET

Tracker	FPS	T. Time	Unique	Avg Obj/F	P	R	F1	MOTP	MOTA	IDSW
Original	2.7299	194.0146	44	10.2399	0.7535	0.3970	0.5200	0.7948	0.2599	66
Custom	14.8748	6.6443	37	8.3919	0.8758	0.3790	0.5291	0.7957	0.3214	35

Trên tập dữ liệu VNTraffic, Custom DeepSORT đạt 14.0458 FPS, cao hơn đáng kể so với 2.8974 FPS của Original DeepSORT. Thời gian xử lý tracker trung bình giảm từ 155.4090 ms xuống 6.6911 ms trên mỗi frame, cho thấy custom tracker nhẹ hơn nhiều so với baseline sử dụng thư viện DeepSORT gốc. Về mức độ khớp bounding box giữa prediction và ground truth, Precision tăng từ 0.8319 lên 0.9257, trong khi Recall giảm nhẹ từ 0.2716 xuống 0.2604. F1-score của hai phương pháp

gần tương đương, lần lượt là 0.4096 và 0.4065. Tuy nhiên, Custom DeepSORT đạt MOTP proxy nhỉnh hơn nhẹ, tăng từ 0.8169 lên 0.8212, cho thấy chất lượng định vị bounding box được duy trì ở mức tương đương hoặc tốt hơn một chút. Ngoài ra, MOTA proxy tăng từ 0.2146 lên 0.2381 và số ID switches giảm từ 35 xuống 23, thể hiện khả năng duy trì định danh ổn định hơn.

Trên tập dữ liệu AICC22-Custom, Custom DeepSORT tiếp tục cho thấy hiệu quả tốt hơn. Processing FPS tăng từ 4.7160 lên 16.7514, trong khi tracker time giảm mạnh từ 96.9407 ms xuống 4.3423 ms trên mỗi frame. Precision tăng từ 0.5546 lên 0.7005, F1-score tăng từ 0.6200 lên 0.6906 và MOTA proxy tăng rõ rệt từ 0.1342 lên 0.3890. MOTP proxy cũng tăng từ 0.8416 lên 0.8583, cho thấy chất lượng khớp bounding box của Custom DeepSORT tốt hơn so với Original DeepSORT. Đặc biệt, số ID switches giảm từ 5 xuống 1, cho thấy rằng custom tracker duy trì định danh phương tiện ổn định hơn trên tập dữ liệu này.

Trên tập dữ liệu CV, Custom DeepSORT tiếp tục đạt tốc độ xử lý cao hơn đáng kể so với Original DeepSORT. Cụ thể, processing FPS tăng từ 2.7299 lên 14.8748, trong khi tracker time giảm mạnh từ 194.0146 ms xuống 6.6443 ms trên mỗi frame. Precision tăng từ 0.7535 lên 0.8758, cho thấy các prediction của Custom DeepSORT có độ chính xác cao hơn. Recall giảm nhẹ từ 0.3970 xuống 0.3790, nhưng F1-score vẫn tăng từ 0.5200 lên 0.5291. MOTP proxy cũng tăng nhẹ từ 0.7948 lên 0.7957, cho thấy chất lượng định vị bounding box được duy trì ổn định. Đặc biệt, MOTA proxy tăng từ 0.2599 lên 0.3214 và số ID switches giảm từ 66 xuống 35, thể hiện Custom DeepSORT duy trì định danh tốt hơn trên tập dữ liệu CV.

Nhìn chung, Custom DeepSORT đạt tốc độ xử lý nhanh hơn rõ rệt và giảm lỗi chuyển đổi ID trên cả ba tập dữ liệu. Kết quả này phù hợp với mục tiêu của hệ thống là xây dựng một tracker nhẹ, dễ giải thích và thân thiện với CPU. Bên cạnh đó, Custom DeepSORT cũng cải thiện MOTP proxy và MOTA proxy, cho thấy mô hình không chỉ nhanh hơn mà còn có chất lượng định vị và độ ổn định tracking tốt hơn. Tuy nhiên, do không sử dụng deep Re-ID, Custom DeepSORT vẫn có thể gặp khó khăn trong các tình huống nhiều phương tiện có màu sắc tương tự nhau hoặc bị che khuất trong thời gian dài.

D. Output Analysis

Ngoài các chỉ số định lượng, hệ thống còn sinh ra các file đầu ra phục vụ quan sát và phân tích. Video đầu ra hiển thị bounding box và track ID của từng phương tiện qua các frame, giúp người dùng kiểm tra trực quan quá trình tracking. File TXT lưu toàn bộ kết quả tracking theo từng frame, thuận tiện cho việc đánh giá lại hoặc phân tích quỹ đạo. File metrics JSON lưu các thông tin thống kê như tổng số frame, FPS, thời gian xử lý, số lượng prediction, số track duy nhất và các chỉ số đánh giá với ground truth.

Đối với video tải lên không có ground truth, hệ thống không tính các chỉ số cần ground truth như Precision, Recall, F1-score, MOTA proxy và MOTP proxy. Thay vào đó, hệ thống báo cáo các thống kê mô tả như số lượng phương tiện được

tracking, số phương tiện trung bình trên mỗi frame, số phương tiện lớn nhất trong một frame, độ dài trung bình của track và tỉ lệ track ngắn. Điều này giúp chế độ Upload vẫn cung cấp được thông tin hữu ích ngay cả khi không có nhãn ground truth.

V. DISCUSSION

Từ kết quả thực nghiệm trên ba tập dữ liệu, Custom DeepSORT đạt tốc độ xử lý cao hơn rõ rệt so với Original DeepSORT. Nguyên nhân chính là custom tracker không sử dụng mạng Re-ID sâu, mà thay bằng đặc trưng histogram màu trong không gian HSV và LAB. Cách tiếp cận này giúp giảm đáng kể thời gian xử lý tracker trên mỗi frame, phù hợp với môi trường CPU và các ứng dụng cần tốc độ gần thời gian thực.

Trên VNTraffic, Custom DeepSORT cải thiện Precision, MOTA proxy và giảm số ID switches từ 35 xuống 23, dù Recall giảm nhẹ so với baseline. Trên AICC22-Custom, Custom DeepSORT cho kết quả tốt hơn ở hầu hết các chỉ số, đặc biệt là FPS, F1-score, MOTA proxy và ID switches. Trên CV, custom tracker cũng đạt FPS cao hơn, tracker time thấp hơn, Precision và F1-score tốt hơn, đồng thời giảm ID switches từ 66 xuống 35. Các kết quả này cho thấy việc kết hợp IoU matching, Kalman Filter, đặc trưng ngoại hình nhẹ và Hungarian assignment có thể duy trì định danh phương tiện khá ổn định trong nhiều bối cảnh khác nhau.

Tuy nhiên, Custom DeepSORT vẫn có một số hạn chế. Do đặc trưng ngoại hình chỉ dựa trên histogram màu, tracker có thể gặp khó khăn khi nhiều phương tiện có màu sắc tương tự, khi ánh sáng thay đổi mạnh hoặc khi đối tượng bị che khuất trong thời gian dài. Ngoài ra, hệ thống vẫn phụ thuộc nhiều vào chất lượng detection của YOLOv5n. Nếu detector bỏ sót hoặc phát hiện sai phương tiện, kết quả tracking cũng có thể bị ảnh hưởng.

Nhìn chung, Custom DeepSORT là một lựa chọn phù hợp cho mục tiêu xây dựng hệ thống theo dõi phương tiện nhẹ, dễ giải thích và có khả năng chạy trên phần cứng hạn chế. So với Original DeepSORT, phương pháp này ưu tiên tốc độ, tính đơn giản và khả năng phân tích từng thành phần thuật toán.

VI. CONCLUSION AND FUTURE SCOPE

Báo cáo đã trình bày một hệ thống theo dõi đa đối tượng cho phương tiện trong video dựa trên YOLOv5n và DeepSORT. Hệ thống sử dụng YOLOv5n để phát hiện các lớp phương tiện như car, motorcycle, bus và truck, sau đó dùng tracker để gán và duy trì ID của từng phương tiện qua các khung hình. Bên cạnh Original DeepSORT, nhóm triển khai Custom DeepSORT với Kalman Filter, IoU matching, đặc trưng histogram màu và thuật toán Hungarian.

Hệ thống được phát triển dưới dạng ứng dụng web, hỗ trợ các chế độ Dataset, Upload, Webcam và Compare. Kết quả thực nghiệm trên VNTraffic, AICC22-Custom và CV cho thấy Custom DeepSORT đạt tốc độ xử lý cao hơn đáng kể, đồng thời giảm số lần chuyển đổi ID trên cả ba tập dữ liệu. Điều này cho thấy một tracker nhẹ và được thiết kế hợp lý vẫn có

thể đạt hiệu quả cạnh tranh trong bài toán theo dõi phương tiện.

Trong tương lai, hệ thống có thể được cải thiện bằng cách sử dụng detector mạnh hơn hoặc fine-tune YOLO trên dữ liệu giao thông thực tế để giảm lỗi bỏ sót. Custom DeepSORT cũng có thể được mở rộng bằng một mô hình Re-ID nhẹ nhằm cải thiện khả năng phân biệt các phương tiện giống nhau. Ngoài ra, hệ thống có thể được tối ưu để chạy trên GPU hoặc thiết bị edge, đồng thời đánh giá thêm bằng các metrics chuẩn của MOTChallenge trên nhiều bối cảnh giao thông phức tạp hơn.

REFERENCES

- [1] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE International Conference on Image Processing (ICIP)*, Phoenix, AZ, USA, 2016, pp. 3464–3468, doi: 10.1109/ICIP.2016.7533003.
- [2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.
- [3] G. Jocher et al., "YOLOv5 by Ultralytics," 2020. [Online]. Available: <https://github.com/ultralytics/yolov5>
- [4] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft COCO: Common objects in context," in *Proc. European Conference on Computer Vision (ECCV)*, Zurich, Switzerland, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.
- [5] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *Proc. IEEE International Conference on Image Processing (ICIP)*, Beijing, China, 2017, pp. 3645–3649, doi: 10.1109/ICIP.2017.8296962.
- [6] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960, doi: 10.1115/1.3662552.
- [7] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955, doi: 10.1002/nav.3800020109.
- [8] K. Bernardin and R. Stiefelhagen, "Evaluating multiple object tracking performance: The CLEAR MOT metrics," *EURASIP Journal on Image and Video Processing*, vol. 2008, Article ID 246309, 2008, doi: 10.1155/2008/246309.
- [9] P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler, and L. Leal-Taixé, "MOT20: A benchmark for multi object tracking in crowded scenes," *arXiv preprint arXiv:2003.09003*, 2020.